CS 330, Fall 2026, Homework 8
Due Wednesday, October 28, 2026

Guidelines. The same homework guidelines apply as for Homework 1.

1. Maximum Clearance Routes

Every year, trucks get stuck on Storrow Drive because of the low bridges (sometimes called "storrowing"). Suppose you're running a trucking company in Boston with n warehouses. Each section of road $e \in E$ will be annotated with the clearance of the lowest bridge on that portion of road c_e. Assume all of the roads are undirected, and that all of the clearances are unique. We'll say that the *clearance of a route* from i to j is the minimum clearance of all the road segments on the route (you can't drive a truck that's taller than the lowest bridge on the route). Next, we'll say that a route is a *maximum-clearance route between i and j* if it is the route between i and j with largest route clearance out of all possible routes from i to j. Intuitively, the maximum-clearance route between two nodes tells you the tallest truck that you can drive between the two locations without getting stuck under a bridge, as well as which path to use.

You want to find an algorithm that computes a tree for max-clearance routes from a given node s. After giving it some thought, you think it might be related to the idea of a maximum spanning tree with respect to the edge clearances.



Figure 1: Storrowed at a perhaps-familiar bridge.

Note: the **maximum** spanning tree of a graph G is just a tree that connects all vertices in G (hence "spanning") while maximizing the sum of the edge weights in the tree. Cycles are not allowed, as we still want a *tree*.

(a) State the cut and cycle property for *maximum* spanning trees (MaxST).

(b) Show that in any graph G with unique edge clearances, the path given by using edges from the maximum spanning tree of G (with respect to the edge weights given by the clearances c_e) gives the maximum clearance route between any two warehouses.

(c) Give a polynomial-time algorithm that takes a graph G with distinct edge clearances and outputs the maximum spanning tree. (*Don't forget the analysis and proof.*)

2. Streaming Edges

(a) Suppose $G = (V, E)$ is an undirected graph with edge weights w, and suppose you have already computed a min-weight spanning tree T for G. Suppose a new edge $e = (u, v)$ with weight $w(e)$ is added to the graph, to obtain a new graph G'. Show how, given $G, T, (u, v)$ and $w(e)$, one can compute a MST T' for the larger graph G' in time $O(n)$ where n is the number of nodes in G. You may assume T is given as a graph with the same node set as G.

Note: $O(n)$ time is not enough to re-read the original graph. Your proof of correctness has to explain why it is OK to ignore lots of the edges in G.

(b) Now consider a setting where you are given the edges of a graph in a *streaming* fashion, that is, you get them one at a time and want to process them using very little additional space. Each edge is described by a triple (u, v, w), where u and v are vertices and w is the weight of the edge. The edges are undirected.

Specifically, you have access to a data structure *edge_stream* that supports the following two operations: *has_next*() and *next_edge*(). A call to *edge_stream.has_next*() returns `true` if there are more edges in the stream and `false` otherwise. *edge_stream.next_edge*() returns the next edge in the stream. There is no way to go backwards and re-read previous edges. (If you want to access edges again, you have to store them.) You may assume that the nodes' names are all integers in $\{1, ..., n\}$.

Your goal is to write an algorithm that computes the minimum-weight spanning tree of the graph in this streaming setting. Write an algorithm that, given a set of vertices V and an edge stream *edge_stream*, builds a min-weight spanning tree of the graph using at most $O(n)$ *space* and $O(n \cdot m)$ time, where $m = |E|$ is the number of edges in *edge_stream*. The stream itself does not count against your space budget, but any data structures or additional storage you use do. [*Hint: Use part 1.*]

An algorithm that uses more than $O(n)$ space will get at most half credit.